

ML-NIDS Ownership Guide

Everything measured, fixed and learned in this project — written so every claim on the page carries the number that backs it. Drill this out loud before an interview; nothing here is asserted without a measurement beside it.

Dataset **CIC-IDS2017** Flows **2,830,628** Features **62** Protocol **temporal / open-set** Defects fixed **9**

CONTENTS

Numbers to know cold The honest result Eight concepts Bug ledger Interview answers What I'd do next

Gotchas

Numbers to know cold

If you can't answer "what's the class balance?" instantly, every metric you quote is uninterpretable. These are the figures an interviewer can check in thirty seconds.

Dataset and split

QUANTITY	VALUE	NOTE
Captures in manifest	8	Mon–Fri, CIC-IDS2017
Rows read	2,830,741	raw CSV
Rows cached	2,830,628	113 dropped: negative <code>Flow Duration</code> , null labels
Features after cleaning	70	78 raw – 7 identifiers – 7 collinear twins
Features the model sees	62	constant columns dropped, fitted on train only
Train rows	1,564,531	Mon–Wed + 50% Thu, de-duplicated (–17.57%)
Validation rows	229,477	50% Thu, never de-duplicated
Test rows	703,198	Friday, never de-duplicated, never touched
Known classes	12	present Mon–Thu
Novel classes	3	Bot, DDoS, PortScan — Friday only

Friday, the test day			
CLASS	COUNT	SHARE	SEEN IN TRAINING?
BENIGN	414,275	58.9%	yes
PortScan	158,930	22.6%	no
DDoS	128,027	18.2%	no
Bot	1,966	0.3%	no
Total	703,198	100%	41.1% unseen

58.91%

all-BENIGN baseline accuracy on Friday

A constant function that ignores its input scores this. Any accuracy figure must be read against it — and the multi-class accuracy of the trained model is *below* it.

Why Thursday is split in half

Thursday is the only day carrying Infiltration and the Web attacks. Hold all of it out and validation contains one class — BENIGN — so macro-F1 is 1.0 by construction and early stopping selects on noise. So Thursday is stratified 50/50 into training and validation. Friday is still never touched by training *or* by threshold calibration.

The honest result

Full XGBoost — 400 rounds, depth 8, balanced sample weights — on the temporal split. Thresholds are quantiles of the **validation** day's benign scores; nothing from Friday enters them.

FPR BUDGET	OBSERVED	DDoS	PORTSCAN	BOT	ANY ATTACK
0.1%	0.14%	61.93%	0.23%	0.00%	27.57%
1.0%	4.38%	63.62%	0.62%	0.05%	28.53%
5.0%	9.48%	67.00%	5.41%	9.41%	32.73%

61.93%

DDoS detection at 0.14% observed false-alarm rate

The one number to lead with. DDoS was never seen during training. Everything else the system catches is a rounding error.

What it actually does

It detects volumetric HTTP floods and essentially nothing else. 62% of DDoS — a class it has never seen — is caught, and those flows are classified as DoS Hulk . That isn't luck: HULK (Wednesday, in training) and LOIC (Friday, unseen) are both high-rate HTTP floods with near-identical flow geometry. The model learned the geometry, not the label. That is real generalisation across attack families, and it's the strongest claim this project supports.

Port scans and botnet C2 are invisible, and that is structural, not a tuning failure. See Concept 6.

Accuracy, and why it is not the headline

READING	VALUE	WHAT IT COUNTS
Multi-class accuracy	58.47%	exact label. Zero attack flows correct — DDoS/PortScan/Bot aren't in the label space
Binary accuracy	69.68%	attack vs benign only
All-BENIGN baseline	58.91%	a constant function

Eleven points apart, same model, same rows. Anyone quoting "accuracy" without saying which has said nothing — and the multi-class figure loses to a constant function, because the model's 0.76% false alarms cost more than its zero correct attack labels gain.

Alert volume — the number that decides deployability

CIC-IDS2017's test day is 41% attack. A real corporate link is nearer 0.1%. Every rate above is measured at roughly 400× the real prevalence, which flatters precision enormously. Projected onto 10M flows/day at 0.1% attack prevalence:

FPR BUDGET	FALSE ALERTS/DAY	TRUE ALERTS/DAY	PRECISION
0.1%	14,252	2,757	16.21%
1.0%	437,532	2,853	0.65%
5.0%	946,997	3,273	0.34%

Loosening the threshold buys almost nothing. Going from 0.1% to 5% multiplies false alerts by 66× and gains 5 points of DDoS. The tight operating point is strictly better — and no operating point here is deployable as-is. Say that; it's the mark of someone who has thought past the leaderboard.

Eight concepts

One evening each, re-derived from scratch with no notes, then repeated. Almost none of this is NIDS trivia — it comes up in every ML interview you will ever sit.

CONCEPT 1 – IMBALANCE

Accuracy is uninterpretable without the class balance

An accuracy figure is only meaningful as a *distance from the trivial baseline*, and the trivial baseline is the class balance. On CIC-IDS2017 overall (>80% benign) a classifier that predicts BENIGN for everything scores >99% accuracy and 0.00 recall on every attack.

The diagnostic test to run on yourself: **name the fix a label implies, then check the fix would work.** "Overfitting" implies regularise, simplify, add data. None of those move attack recall here — so the word is wrong. A diagnosis implying no working treatment is not a diagnosis.

SAY THIS

"Accuracy on this dataset is meaningless — 59% of my test day is benign, so a constant function scores 58.91%. I report per-class detection rate at a fixed false-alarm budget instead."

CONCEPT 2 – SPLITS

Temporal splits vs random splits

A random split over a week of bursty network traffic is optimistic: flows from the same attack burst are near-identical and land on both sides, so the model is graded on rows it has effectively memorised. The temporal protocol trains on Mon–Wed + half Thursday and tests on Friday, whose DDoS, PortScan and Bot are genuinely absent from training.

The gap is not subtle: the same repository's closed-set random-split comparison reports 0.9988 accuracy. The temporal split reports 58.47%. **Same data, same features — the split protocol is worth 40 points.**

SAY THIS

"I evaluate on a temporal split because a random split over bursty traffic leaks. Reporting it cost me forty points of accuracy, which is roughly how much the random split was flattering me."

CONCEPT 3 – LEAKAGE

Fit on train, transform everywhere

Three statistics can leak if fitted on the whole dataset: the imputation median, the >50%-missing column threshold, and the constant-column list. All three now live in a `FrameCleaner` with `fit / transform`, fitted on the training split only. The `StandardScaler` likewise. Three regression tests pin it.

The subtler form: **de-duplication is a protocol-dependent decision and must not live in a shared cache**. For a random split, identical rows on both sides are contamination and dedup is correct. For a temporal split, a Friday flow identical to a Monday flow is not a leaked row — it's the same traffic pattern recurring, which is exactly what happens in deployment. Dedup now runs inside `build_splits`, on training only.

SAY THIS

"You may de-duplicate a training set. You may never de-duplicate a test set — it makes the test set's composition a function of what happened to be in training."

CONCEPT 4 – OPEN SET

A closed-set posterior cannot say "none of the above"

Softmax over K known classes always sums to 1. Asked about a class it has never seen, the model must still distribute all its mass across the K it knows — it can be *wrong*, never *uncertain about the whole label space*. Tree ensembles make it worse: leaves at the edge of the training hull are the purest, so out-of-distribution points get *higher* confidence.

This is why the binary ATTACK/BENIGN framing collapsed to 0.085% attack recall on Friday. Squashing everything to one ATTACK label told the model "attack is one coherent class." It isn't. The model learned *"attack looks like Wednesday's DoS Hulk"*, and on Friday's different geometry it correctly concluded "not the attack I know" — which the binary framing forces it to express as BENIGN.

SAY THIS

"It's an open-set problem. A closed-set posterior can't express 'none of the above', so I attach a separate novelty score and reject at a threshold calibrated on validation."

CONCEPT 5 – REJECTION

Rejection scoring and threshold calibration

Four scorers, all "higher = more novel": max-softmax (the weak baseline), energy $-T \cdot \log \text{sumexp}(z/T)$, Mahalanobis distance to the nearest class-conditional Gaussian under a Ledoit–Wolf shrunk shared covariance, and reconstruction error. Mahalanobis is the only option for a Random Forest, which has no logits.

0.657 AUROC, Mahalanobis on scaled features, known vs unknown

TPR@5%FPR = 1.69%. Weak, and honestly so. The `config.py` claim of 0.914 is for `mahalanobis_embed` — distance in a trained neural encoder's embedding space, a different scorer whose encoder does not exist in this repository yet. Know which one your number is.

τ is always a quantile of the **validation** scores. Nothing from the test day may enter it. And an AUROC below 0.5 is not noise — it means real signal with the sign reversed. The fix is a scorer that measures distance from the training manifold, never flipping the sign, which is fitting to test labels.

SAY THIS

"AUROC 0.657 with TPR of 1.7% at a 5% false-positive rate. It's weak — the rejector is where the remaining headroom is, and I'd attack it before touching the classifier."

CONCEPT 6 – TRAIN/SERVE SKEW

Port scans have no per-flow signature — by construction

A port scan is the same flow repeated against different ports: identical duration, packet counts, lengths and TCP flags. The only thing distinguishing one scan flow from another is the destination port — which is deliberately dropped as a shortcut feature (otherwise a tree learns "port 21 → FTP-Patator" and nothing about traffic).

1,958 unique feature vectors among 158,930 PortScan flows

Without `Destination Port` in the key. With it: 90,819. So **98.8% of what distinguishes one scan flow from another lives in the port alone.**

Scanning is a property of a set of flows — one source touching many ports in a window — not of any one flow. A per-flow classifier is structurally incapable of seeing it, and three independent experiments confirm capacity is not the constraint: a 25-tree RF, a 400-round balanced XGBoost, and every threshold between 0.1% and 5% FPR all return the same near-zero.

Two decisions here are individually correct and catastrophic in composition: dropping the port (right), then de-duplicating on the remaining features (right) deleted 98.8% of the PortScan class from the dataset. Nothing in the code made the composition visible. **That is what an emergent bug is, and why every individual code review passes.**

SAY THIS

"My per-flow classifier catches 0.2% of port scans, and I can show you why that's not a tuning problem — 158,930 scan flows reduce to 1,958 unique feature vectors once the port is removed. Scanning is a set-level property. The fix is windowed aggregation per source IP, not a better model."

CONCEPT 7 – BASE RATES

The base-rate fallacy decides deployability

A 0.76% false-alarm rate sounds excellent. On 10M flows/day at 0.1% attack prevalence it is **76,000 false alerts per day**, against roughly 6,000 true ones — twelve false alarms for every real detection. No SOC consumes that.

The asymmetry to internalise: **changing the base rate wrecks precision and leaves recall untouched**. Recall is computed within the positive class, so prevalence can't move it. Precision depends on the ratio of positives to negatives, so it collapses as attacks get rarer. Every benchmark number you will ever read was measured at a prevalence the deployment will not have.

SAY THIS

"My test day is 41% attack; a real link is 0.1%. That's 400× the prevalence, so my precision numbers are flattered by roughly that factor. Recall transfers; precision doesn't."

CONCEPT 8 – CALIBRATION

A good ranker is not a good probability estimator

Measured on benign rows, validation versus test:

MODEL	SPLIT	Q50	Q90	Q99	Q99.9
XGBoost	Val (Thu)	0.00001	0.00011	0.00487	0.34992
XGBoost	Test (Fri)	0.00001	0.00034	0.11339	0.38007
Random Forest	Val (Thu)	0.00062	0.06744	0.32252	0.48513
Random Forest	Test (Fri)	0.00005	0.08000	0.32320	0.49458

XGBoost's q99 moves **23×** between days; the Random Forest's moves 0.2%. So a threshold targeting 1% false alarms lands at 4.38% for XGBoost and 1.02% for RF. **A threshold that misses its target by 4× is not an operating point, it's a guess.**

Why: boosting optimises a loss, not calibration, and `compute_sample_weight("balanced")` makes it far worse by inflating rare-class gradients — benign probabilities get crushed toward zero, so the quantile sits on a near-vertical stretch of the CDF where a tiny distribution shift moves the rate enormously. An RF

probability is a vote fraction averaged over trees: inherently granular and smooth. **XGBoost is the better ranker and the worse probability estimator.** Fix with isotonic calibration on validation, or threshold by rank rather than by value.

SAY THIS

"XGBoost ranks better but its probabilities aren't calibrated, so my thresholds didn't transfer across days — a 1% budget came out at 4.4%. Random Forest's transferred to within 0.02 points. I'd wrap the classifier in isotonic calibration fitted on the validation day."

Also worth knowing, one level down: hypersphere collapse in Deep SVDD — any learnable additive term (a `Linear` bias, an affine `BatchNorm`) lets the encoder output a constant and drive the objective to zero (Ruff et al., ICML 2018, Prop. 2). Early stopping must be *off*: a lower validation distance is the signature of collapse, not of a better model.

Bug ledger

Nine defects found by running the code rather than reading it. Each one is an interview answer; the lesson matters more than the fix.

B1 Refactor changed a return type; neither call site was updated

`predict()` was changed to return a dict. Both callers still did `attack, confidence = predict(features)` — `ValueError` on every call, so `/predict` returned 500 and the live worker stored nothing. Worse, a broad `except Exception` in the cleanup thread swallowed it, so instead of crashing it logged one error per flow forever. **Lesson: a signature change's blast radius is every call site — and a broad except turns a loud crash into a silent lie.**

B2 Two bugs stacked; the outer one hid the inner one

The model bundle failed to load first, so `predict()` raised before ever reaching the unpacking error. Fixing the model would have surfaced a **different** 500. Diagnosing it required stubbing a fake bundle to make the second bug reachable. **Lesson: when a failure masks a failure, stub past the outer one deliberately.**

B3 A download button that served a stale file

`GET /download` streamed `predicted_output.csv` off the server's working directory — a file the service had deliberately stopped writing. Every click returned a report from two months earlier regardless of what had just been uploaded. **Lesson: silently wrong beats loudly broken, and is far more dangerous. Per-request state does not belong in a fixed file on disk.**

B4 An integrity check that compared counts, not names

`Bundle.check()` verified that the scaler width, the model's `n_features_in_` and `len(feature_names)` agreed. A stored artifact had 70 features and the current pipeline produced 70 — but the lists were **disjoint in eight positions each way**, and one contained `Destination Port`. It would have loaded cleanly and served confidently wrong predictions forever. **Lesson: verify identity, not arity.**

B5 The rejector was measured and then thrown away

Training fitted a `MahalanobisScorer`, calibrated τ , printed the detection metrics — and never saved either. The serving layer looked for them in `manifest.json`, and a fitted estimator cannot survive a JSON round-trip, so the novelty branch was unreachable code. Every served prediction came back with `is_known=None`. **Lesson: a rejector is part of the model, not part of the report.**

B6 One OOM habit in four different functions

`build_cache` held 8 frames, concatenated (copy 2), then defensively copied (copy 3) — peak ~4× the dataset, OOM-killed on 863 MB of CSV. Fixing it moved the crash to `load_clean`; fixing that moved it to `_finalise`; fixing that surfaced `MahalanobisScorer.fit`, which made four float64 copies for a covariance estimate. **Lesson: it wasn't a bug, it was a habit — "materialise everything, then transform it". Whack-a-mole is what fixing symptoms looks like.**

B7 De-duplication deleted 98.8% of an attack class

Dropping `Destination Port` (correct) then de-duplicating on features (correct) reduced `PortScan` from 158,930 to 1,851 rows, and moved the benign baseline from 58.9% to 72.3%. `keep="first"` in manifest order also meant Monday kept everything and Friday lost 72,232 benign flows — making the test set's composition a function of the training set. **Lesson: emergent bugs pass every individual code review.**

B8 A dashboard that stated a metric it did not read

`ModelInfo.jsx` hardcoded "Validated Accuracy 99.83%", "78 Dimensions" and "15 Categories". Nothing in the repository produced any of them; the loaded model has 62 features and 12 classes. `/health` likewise returned `{"status": "Healthy"}` unconditionally and stayed green while every prediction 500'd. **Lesson: a UI that states a metric must read it from the artifact that earned it. A health check that cannot go red is decoration.**

B9 Artifacts welded to a library version

`deep_svdd/feature_names.pkl` raises `NotImplementedError` on load — someone pickled a pandas `Index` instead of a plain list, so it only opens under the pandas version that wrote it. Pickled sklearn estimators warn the same way across versions. **Lesson: artifacts store primitives, never library objects whose internals you don't control.**

Interview answers

Say these out loud until they're automatic. Writing an answer and speaking one are different skills, and only one gets tested.

Q Walk me through this project in 90 seconds.

"It's a network intrusion detection system that does live inference, not just offline scoring. Scapy captures packets, a flow manager reassembles them into bidirectional flows keyed on the 5-tuple, I extract CICFlowMeter-compatible features, and a gradient-boosted model classifies each completed flow. FastAPI serves it, React dashboards it, SQLite logs it. It runs standalone on one host — that constraint drove the storage and auth decisions.

The part worth talking about is the evaluation. CIC-IDS2017 is ~59% benign on my test day, so accuracy is nearly meaningless. I split it temporally — Monday to Wednesday plus half of Thursday for training, the other half for validation, Friday for test — and Friday holds DDoS, PortScan and Bot, which the model has never seen. That makes it an open-set problem, so a closed-set softmax can't express 'none of the above'; I attach a Mahalanobis novelty score with a threshold calibrated on validation only.

The honest headline: **62% detection on unseen DDoS at a 0.14% false-alarm rate.** Port scans are 0.2%, and I can explain exactly why that's structural."

THE TRAP

Do not open with what you wanted to learn, or with a list of algorithms you tried. "I used LR, DT, RF and XGBoost to get optimal results" reads as running the sklearn model zoo and picking the best test number. Lead with what the system does and one defensible figure.

Q Give me one number for your resume line, and tell me exactly what it measures.

"62% detection rate on DDoS at a 0.14% false-alarm rate — full XGBoost, 400 rounds, on the temporal cross-day protocol, where DDoS is a class absent from training. The threshold is a quantile of the validation day's benign scores; nothing from the test day enters it."

THE TRAP

Never quote a number you cannot produce on demand. Every figure on this page is written to `runs/<sha>-<timestamp>/`. If you can't point at the file, don't say the number.

Q Your classification report shows 0.00085 recall on ATTACK. What happened?

"Majority-class collapse under distribution shift — not overfitting. Regularisation, more data and a simpler model would all fail to fix it, which is how I know the label is wrong.

That was a binary ATTACK/BENIGN model on the cross-day split. Squashing every attack into one label told the model 'attack is one coherent class'. It isn't. It learned the geometry of Wednesday's DoS Hulk, because that's the only attack it saw enough of, and on Friday's different geometry it correctly concluded 'not the attack I know' — which the binary framing forces it to express as BENIGN. Accuracy came out at 0.6415, which is the benign share to four decimal places: the model's entire contribution over a constant function was 32 net correct flows out of 615,848."

Q Can you get this above 80%?

"Which 80%? Multi-class and binary accuracy differ by eleven points on this split, and that's not pedantry."

On binary: no, not by modelling. I did the ceiling analysis — a *flawless* classifier on BENIGN and DDoS, leaving scans and bots at zero, caps at 77.14%. You cannot reach 80% without PortScan, and PortScan has no per-flow signature. On multi-class the question is malformed: DDoS, PortScan and Bot aren't in the label space, so the model can't emit them. The correct target for those rows is `Unknown_Attack` via rejection — which is where the real headroom is, since my rejector is at AUROC 0.657."

WHY THIS ANSWER WINS

Almost every candidate says "more tuning" or "more data". Doing the ceiling analysis first — and being able to say *which* class holds the headroom and why it's unreachable — is what separation looks like.

Q What's the hardest bug you've fixed here?

"A refactor changed my prediction service's return type from a tuple to a dict, and neither call site was updated. What made it interesting is that two bugs were stacked: the model bundle also failed to load, so the service raised before ever reaching the unpacking error. The outer failure masked the inner one — I had to stub a fake bundle to make the second reachable. And in the live worker a broad `except Exception` swallowed it, so instead of crashing it logged one error per flow and silently stored nothing; the dashboard just showed zeros. I fixed both call sites and pinned it with an AST-level test that fails if any file unpacks that function — then verified the test by reintroducing the bug and watching it fail."

Q How did you build this?

"I built it with AI agents, and the most interesting part was where they failed. My knowledge base went 40% stale within eight hours of active development because the docs asserted behaviour the refactor had deleted. A hook I'd written looked correct and was

completely inert — matches for the wrong runtime. And a refactor updated a function's return type without touching either caller, which broke every prediction silently because a broad except swallowed it. I can walk you through how I'd catch each of those in CI."

ON HONESTY

Nobody asks who typed the characters. The risk is claiming ownership of something you can't defend. The way through is to know the code cold — then "I built this" is true in every sense an interviewer cares about.

What I'd do next

Often the last question in a loop, and the one that separates people. Ranked by honest expected value, not by interest.

1. Windowed aggregation for scan detection

Count distinct destination ports per source IP per 60-second window. Port scanning is arguably the *easiest* attack to detect — just not at the flow level. Worth the entire 22.6 points of Friday that the per-flow model cannot reach.

One nuance worth saying out loud: the ML-ready CIC-IDS2017 CSVs strip `Source IP`, so this cannot be benchmarked offline on this dataset. But the live system has it — `flow_manager.py` already tracks `src_ip` — so it is implementable in the product and demonstrable live. Knowing the difference between "can't build it" and "can't benchmark it" reads as deployment experience.

2. Calibrate the classifier's probabilities

`CalibratedClassifierCV(method="isotonic", cv="prefit")` fitted on the validation day, then threshold the calibrated output. Directly fixes the 4× threshold miss measured in Concept 8. Cheap, and it makes every operating point mean what it says.

3. Improve the rejector

Where the theoretical headroom is, and the most open-ended. Options: per-class covariance instead of one shared matrix; Mahalanobis on a trained MLP's penultimate embeddings; τ recalibrated per day rather than baked into a pickle. Research-shaped — no guaranteed payoff.

4. Two-stage detection

Binary attack/benign first, then family classification on the positives. The binary framing already counts DDoS→DoS Hulk as correct, which it should — the analyst cares that it fired, not which flood tool it was.

Gotchas

Things that cost real time. All of them are the kind of detail that makes a demo fail in front of an interviewer.

- **Never rename a folder containing a virtualenv.** Windows bakes the absolute interpreter path into every `.exe` launcher in `Scripts\`; on Linux the `bin/python` symlink and `pyvenv.cfg` break the same way. `python` keeps working, `pip / uvicorn / pytest` do not. Rebuild, don't relocate — and `python -m pip` bypasses a broken launcher in a pinch.
- **Check which python is first on PATH.** An MSYS2 Python 3.14 shadowing the Windows CPython 3.11 produces a venv with `bin/` instead of `Scripts/`, and wheels that will not install. `where.exe python`.
- **Pin pandas deliberately.** pandas 3.0 changed the default string dtype away from `object`, so `if df[c].dtype == object` silently stops firing. Ask `pd.api.types.is_numeric_dtype` what the dtype *is*, not what it used to be called.
- **The captures are not all the same schema.** `Friday-...-DDos.csv` ships 85 columns from the GeneratedLabelledFlows distribution; the other seven ship 79. It's currently harmless only because all six extras are identifiers that get dropped — luck, not design. The loader now asserts the invariant.
- **A test suite that cannot run is not a test suite.** All 31 original tests imported torch at module scope, so in any environment without it every one failed at collection and nothing in `backend/` was covered. Split tests by dependency.
- **Verify a guard by firing it.** Break the thing once and watch the test fail, then revert. A test you have never seen fail is a decoration.

All figures measured on this repository between 11 and 12 September 2026, on the crossday protocol unless stated. Detection tables regenerate to `runs/<git-sha>-<timestamp>/detection_by_class.csv` via `python src/run.py train --model xgb --protocol crossday`. Where a number here disagrees with a future run, the run wins — and that is the point.